BIG DATA ANALYTICS LAB EXAM
FULL PROGRAMS AND COMMANDS

Each experiment follows: Aim, Procedure, Program/Commands, Output, Result.

Important: Java MapReduce examples require a Hadoop installation and Hadoop libraries on the compile classpath. Replace local file paths with paths available on your lab computer. Hive/HBase/Thrift commands may vary by installed version. The supplied lab sheet names the experiments but does not provide source code or exact software versions.

Experiment 1: Install Hadoop, Configure Files, and Start Services

Aim

To install Hadoop, configure the main XML files, and start HDFS and YARN.

Procedure

1. Install a compatible Java version and extract Hadoop.
2. Set JAVA_HOME, HADOOP_HOME, and PATH.
3. Configure core-site.xml, hdfs-site.xml, mapred-site.xml, and yarn-site.xml.
4. For a new single-node setup only, format the NameNode.
5. Start HDFS and YARN and verify the Java processes.

Program / Commands

```
# Linux shell commands (example; use a compatible Java/Hadoop version)
java -version
tar -xzf hadoop-<version>.tar.gz
export HADOOP_HOME=$HOME/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export JAVA_HOME=/path/to/java

# $HADOOP_HOME/etc/hadoop/core-site.xml
<configuration>
  <property><name>fs.defaultFS</name><value>hdfs://localhost:9000</value></property>
</configuration>

# hdfs-site.xml (single-node example)
<configuration>
  <property><name>dfs.replication</name><value>1</value></property>
  <property><name>dfs.namenode.name.dir</name><value>file:///home/student/hadoopdata/namenode</value></
property>
  <property><name>dfs.datanode.data.dir</name><value>file:///home/student/hadoopdata/datanode</value></
property>
</configuration>

# mapred-site.xml
<configuration>
  <property><name>mapreduce.framework.name</name><value>yarn</value></property>
</configuration>

# yarn-site.xml
<configuration>
  <property><name>yarn.nodemanager.aux-services</name><value>mapreduce_shuffle</value></property>
</configuration>

# Format only for a new single-node setup; formatting an existing cluster can destroy HDFS metadata
```

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
jps
```

Output

The jps output should show services such as NameNode, DataNode, ResourceManager, and NodeManager, depending on configuration.

Result

Thus, Hadoop was configured and its services were started.

Experiment 2: Matrix Multiplication Using Hadoop MapReduce

Aim

To multiply matrices A and B using Hadoop MapReduce.

Procedure

6. Save the Java source as MatrixMultiply.java.
7. Prepare input records in the format A,row,col,value or B,row,col,value; row and column indexes are zero-based.
8. Compile with Hadoop libraries, package the classes into a JAR, and upload input to HDFS.
9. Run with the number of rows in A and columns in B.
10. Read the output part file.

Program / Commands

```
// MatrixMultiply.java
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MatrixMultiply {
  public static class M extends Mapper<LongWritable, Text, Text, Text> {
    public void map(LongWritable key, Text value, Context ctx)
        throws IOException, InterruptedException {
      // Input format: A,row,col,value OR B,row,col,value
      String[] p = value.toString().trim().split(",");
      if (p.length != 4) return;
      String matrix = p[0].trim();
      int r = Integer.parseInt(p[1].trim());
      int c = Integer.parseInt(p[2].trim());
      String v = p[3].trim();
      int rowsA = ctx.getConfiguration().getInt("matrix.rowsA", 2);
      int colsB = ctx.getConfiguration().getInt("matrix.colsB", 2);
      if (matrix.equals("A")) {
        for (int j = 0; j < colsB; j++)
          ctx.write(new Text(r + "," + j), new Text("A," + c + "," + v));
      } else if (matrix.equals("B")) {
        for (int i = 0; i < rowsA; i++)
          ctx.write(new Text(i + "," + c), new Text("B," + r + "," + v));
      }
```

```
    }
  }
  public static class R extends Reducer<Text, Text, Text, IntWritable> {
    public void reduce(Text key, Iterable<Text> vals, Context ctx)
        throws IOException, InterruptedException {
      Map<Integer,Integer> a = new HashMap<>();
      Map<Integer,Integer> b = new HashMap<>();
      for (Text t : vals) {
        String[] p = t.toString().split(",");
        int k = Integer.parseInt(p[1]);
        int v = Integer.parseInt(p[2]);
        if (p[0].equals("A")) a.put(k, v); else b.put(k, v);
      }
      int sum = 0;
      for (Integer k : a.keySet())
        if (b.containsKey(k)) sum += a.get(k) * b.get(k);
      ctx.write(key, new IntWritable(sum));
    }
  }
  public static void main(String[] args) throws Exception {
    if (args.length != 4) {
      System.err.println("Usage: MatrixMultiply <input> <output> <rowsA> <colsB>");
      System.exit(2);
    }
    Configuration conf = new Configuration();
    conf.setInt("matrix.rowsA", Integer.parseInt(args[2]));
    conf.setInt("matrix.colsB", Integer.parseInt(args[3]));
    Job job = Job.getInstance(conf, "Matrix Multiplication");
    job.setJarByClass(MatrixMultiply.class);
    job.setMapperClass(M.class); job.setReducerClass(R.class);
    job.setMapOutputKeyClass(Text.class); job.setMapOutputValueClass(Text.class);
    job.setOutputKeyClass(Text.class); job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
# Example input.txt for A=[[1,2],[3,4]], B=[[5,6],[7,8]]
A,0,0,1
A,0,1,2
A,1,0,3
A,1,1,4
B,0,0,5
B,0,1,6
B,1,0,7
B,1,1,8

# Build/run (classpath command may vary by Hadoop distribution)
hadoop com.sun.tools.javac.Main MatrixMultiply.java
jar cf matrix.jar MatrixMultiply*.class
hdfs dfs -mkdir -p /lab/matrix-in
hdfs dfs -put input.txt /lab/matrix-in/
hadoop jar matrix.jar MatrixMultiply /lab/matrix-in /lab/matrix-out 2 2
hdfs dfs -cat /lab/matrix-out/part-r-00000
```

Output

Expected values (keys are row,column):

0,0 19

0,1 22

1,0 43
1,1 50

Result

Thus, the product matrix was computed using MapReduce.

Experiment 3: HDFS: Create Directories and Add Files

Aim

To create an HDFS directory and upload files.

Procedure

11. Start HDFS.
12. Create the destination directory.
13. Upload a local file.
14. List the HDFS directory.

Program / Commands

```
hdfs dfs -mkdir -p /user/student/input
hdfs dfs -put sample.txt /user/student/input/
hdfs dfs -ls /user/student/input
hdfs dfs -cat /user/student/input/sample.txt
```

Output

The listing shows sample.txt and cat displays its contents.

Result

Thus, a file and directory were created in HDFS and verified.

Experiment 4: Count Characters in Each Word Using MapReduce

Aim

To output the character length of each word in the input.

Procedure

15. Save the source as WordCharacterCount.java.
16. Upload a text file to HDFS.
17. Compile the source and create a JAR.
18. Run the job and view the reducer output.

Program / Commands

```
// WordCharacterCount.java
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCharacterCount {
  public static class MapClass extends Mapper<LongWritable, Text, Text, IntWritable> {
```

```
    private final Text word = new Text();
    private final IntWritable length = new IntWritable();
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
      for (String token : value.toString().split("\\\\s+")) {
        String w = token.replaceAll("[^A-Za-z0-9]", "");
        if (!w.isEmpty()) {
          word.set(w); length.set(w.length());
          context.write(word, length);
        }
      }
    }
  }
  public static class ReduceClass extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
      // Output each word's character length once; repeated words may appear more than once.
      for (IntWritable value : values) context.write(key, value);
    }
  }
  public static void main(String[] args) throws Exception {
    if (args.length != 2) {
      System.err.println("Usage: WordCharacterCount <input> <output>");
      System.exit(2);
    }
    Job job = Job.getInstance(new Configuration(), "Character count per word");
    job.setJarByClass(WordCharacterCount.class);
    job.setMapperClass(MapClass.class); job.setReducerClass(ReduceClass.class);
    job.setMapOutputKeyClass(Text.class); job.setMapOutputValueClass(IntWritable.class);
    job.setOutputKeyClass(Text.class); job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
hadoop com.sun.tools.javac.Main WordCharacterCount.java
jar cf wordchar.jar WordCharacterCount*.class
hadoop jar wordchar.jar WordCharacterCount /lab/input /lab/wordchar-out
hdfs dfs -cat /lab/wordchar-out/part-r-00000
```

Output

For input “big data”, output is:
big 3
data 4

Result

Thus, the number of characters in each word was counted.

Experiment 5: Upload Files to HDFS

Aim

To copy a local file to HDFS.

Procedure

19. Create a target directory.
20. Use put or copyFromLocal to upload.

21. Verify using ls.

Program / Commands

```
hdfs dfs -mkdir -p /user/student/files
hdfs dfs -put sample.txt /user/student/files/
# Alternative:
hdfs dfs -copyFromLocal sample.txt /user/student/files/
hdfs dfs -ls /user/student/files
```

Output

The directory listing shows sample.txt.

Result

Thus, the local file was uploaded to HDFS.

Experiment 6: Word Count Using Hadoop MapReduce

Aim

To count the frequency of every word in a text file.

Procedure

22. Save as WordCount.java.
23. Upload the input file to HDFS.
24. Compile and package the Java class.
25. Run the job and read the output.

Program / Commands

```
// WordCount.java
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
  public static class TokenizerMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    private static final IntWritable ONE = new IntWritable(1);
    private final Text word = new Text();
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
      for (String token : value.toString().toLowerCase().split("\\\\s+")) {
        String w = token.replaceAll("[^a-z0-9]", "");
        if (!w.isEmpty()) { word.set(w); context.write(word, ONE); }
      }
    }
  }
  public static class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable v : values) sum += v.get();
      context.write(key, new IntWritable(sum));
    }
```

```
  }
  public static void main(String[] args) throws Exception {
    if (args.length != 2) {
      System.err.println("Usage: WordCount <input> <output>");
      System.exit(2);
    }
    Job job = Job.getInstance(new Configuration(), "Word Count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class); job.setCombinerClass(SumReducer.class);
    job.setReducerClass(SumReducer.class);
    job.setOutputKeyClass(Text.class); job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
hadoop com.sun.tools.javac.Main WordCount.java
jar cf wordcount.jar WordCount*.class
hadoop jar wordcount.jar WordCount /lab/input /lab/wordcount-out
hdfs dfs -cat /lab/wordcount-out/part-r-00000
```

Output

Input: big data big
Output:
big 2
data 1

Result

Thus, word frequency was calculated using Hadoop MapReduce.

Experiment 7: Delete Files from HDFS

Aim

To delete a file or directory from HDFS.

Procedure

26. Check the target path.
27. Delete a file with rm.
28. Use rm -r for a directory and its contents.
29. Verify with ls.

Program / Commands

```
hdfs dfs -ls /user/student/input
hdfs dfs -rm /user/student/input/sample.txt
# Recursively delete a directory:
hdfs dfs -rm -r /user/student/oldfolder
```

Output

HDFS reports the deleted path; the next listing no longer shows it.

Result

Thus, HDFS file deletion was performed.

Experiment 8: Install Hive and Start the Shell

Aim

To configure Hive and open the Hive command-line interface.

Procedure

30. Install compatible Java and Hadoop and start HDFS.
31. Download and extract Apache Hive.
32. Set HIVE_HOME and PATH.
33. Configure the metastore and hive-site.xml for the selected Hive version.
34. Initialize the metastore schema when needed, then launch Hive.

Program / Commands

```
export HIVE_HOME=$HOME/apache-hive
export PATH=$PATH:$HIVE_HOME/bin
hadoop version
# For a new Derby metastore only; do not repeat on an existing metastore:
schematool -dbType derby -initSchema
hive
```

Output

The Hive prompt appears, commonly hive>. Exact setup depends on Hive version and metastore settings.

Result

Thus, Hive was configured and launched.

Experiment 9: Hive: Create Employees Table, Load Data, Display Contents

Aim

To create an employees table, load data, and display its rows.

Procedure

35. Create a tab-separated local file employees.txt.
36. Create the table with the required four fields.
37. Load the file into the table.
38. Run SELECT.

Program / Commands

```
-- employees.txt fields are tab-separated:
-- 101<TAB>Ravi<TAB>21<TAB>CSE
-- 102<TAB>Priya<TAB>22<TAB>AIML

CREATE TABLE employees (
  eid INT, ename STRING, eage INT, edepartment STRING
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
STORED AS TEXTFILE;

LOAD DATA LOCAL INPATH '/home/student/employees.txt'
INTO TABLE employees;
SELECT * FROM employees;
```

Output

Example output contains rows 101 Ravi 21 CSE and 102 Priya 22 AIML.

Result

Thus, the employee table was created, loaded, and displayed.

Experiment 10: Install HBase and Start Services

Aim

To configure HBase and start the HBase services.

Procedure

39. Install compatible Java and Hadoop if using HBase with HDFS.
40. Download and extract HBase.
41. Set HBASE_HOME and PATH.
42. Configure hbase-site.xml for standalone or distributed mode.
43. Start HBase and open the shell.

Program / Commands

```
export HBASE_HOME=$HOME/hbase
export PATH=$PATH:$HBASE_HOME/bin
# Configure $HBASE_HOME/conf/hbase-site.xml for your environment
start-hbase.sh
jps
hbase shell
```

Output

The HBase shell prompt opens; HMaster and RegionServer may appear in jps depending on configuration.

Result

Thus, HBase was configured and started.

Experiment 11: Install Thrift, Generate HBase Bindings, and Interact

Aim

To generate Thrift client bindings and access HBase through the Thrift service.

Procedure

44. Install a Thrift compiler compatible with the HBase release.
45. Locate the matching HBase Thrift IDL file.
46. Generate client code for the selected language.
47. Start the Thrift service using the version-specific command.
48. Run a generated client against the configured host and port.

Program / Commands

```
thrift --version
# Run from the directory containing the matching HBase Thrift IDL:
thrift --gen java Hbase.thrift
# Start the Thrift service using the command supported by your HBase version.
# Run the generated Java client configured with the service host and port.
```

Output

The generated client should connect and return the requested HBase result. The precise service command and API depend on HBase/Thrift versions.

Result

Thus, Thrift bindings were generated and used to interact with HBase.

Experiment 12: HBase: Create Student Table, Insert, Get, Delete

Aim

To create a student table, insert values, retrieve them, and delete data.

Procedure

49. Open hbase shell.
50. Create a table with two column families.
51. Insert values with put.
52. Retrieve values with get.
53. Delete a cell or the full row.

Program / Commands

```
create 'student', 'personal', 'academic'
put 'student', '101', 'personal:name', 'Ravi'
put 'student', '101', 'academic:department', 'AIML'
get 'student', '101'
delete 'student', '101', 'personal:name'
scan 'student'
# To delete the complete row instead:
deleteall 'student', '101'
scan 'student'
```

Output

get displays row 101 and its stored values. After deleteall, scan no longer returns that row.

Result

Thus, HBase create, insert, get, and delete commands were executed.

Experiment 13: Hive DDL Commands

Aim

To demonstrate Hive Data Definition Language commands.

Procedure

54. Create and select a database.
55. Create a table.
56. Inspect databases, tables, and schema.
57. Alter the table and drop objects.

Program / Commands

```
CREATE DATABASE labdb;
SHOW DATABASES;
USE labdb;
```

```
CREATE TABLE student (sid INT, sname STRING);
SHOW TABLES;
DESCRIBE student;
ALTER TABLE student ADD COLUMNS (department STRING);
DROP TABLE student;
DROP DATABASE labdb;
```

Output

SHOW and DESCRIBE display object details; DROP removes the specified objects.

Result

Thus, Hive DDL commands were executed.

Experiment 14: Hive DML Commands

Aim

To insert and retrieve data using Hive DML commands.

Procedure

58. Create a table.
59. Insert sample records.
60. Select all records.
61. Use LOAD DATA if a local data file is supplied.

Program / Commands

```
CREATE TABLE staff (
  eid INT, ename STRING, department STRING
)
STORED AS ORC;
INSERT INTO staff VALUES
(1, 'Ravi', 'CSE'),
(2, 'Priya', 'AIML');
SELECT * FROM staff;
```

Output

The output displays the two inserted staff rows.

Result

Thus, Hive insertion and retrieval commands were executed.

Experiment 15: Hive CREATE, SHOW, DESCRIBE, USE, DROP, ALTER, TRUNCATE

Aim

To execute common Hive database and table commands.

Procedure

62. Create a database and select it.
63. Create and describe a table.
64. Alter the table structure.
65. Insert a sample row, truncate the table, and drop the objects.

Program / Commands

```
CREATE DATABASE labdb;
USE labdb;
CREATE TABLE employee (eid INT, ename STRING);
SHOW TABLES;
DESCRIBE employee;
ALTER TABLE employee ADD COLUMNS (department STRING);
INSERT INTO employee VALUES (1, 'Ravi', 'CSE');
TRUNCATE TABLE employee;
SELECT * FROM employee;
DROP TABLE employee;
DROP DATABASE labdb;
```

Output

SHOW lists employee; DESCRIBE shows its schema; SELECT after TRUNCATE returns no rows.

Result

Thus, the specified Hive commands were executed.

Experiment 16: Multiply Two Numbers Using Hadoop MapReduce

Aim

To multiply two numbers using a MapReduce job. The reducer multiplies all numeric values in the input; provide exactly two numbers for this experiment.

Procedure

66. Save as NumberProduct.java.
67. Create a text file containing one number per line.
68. Upload it to HDFS.
69. Compile, package, and run the job.
70. Read the output.

Program / Commands

```
// NumberProduct.java
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class NumberProduct {
  public static class NumberMapper extends Mapper<LongWritable, Text, Text, LongWritable> {
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
      String s = value.toString().trim();
      if (!s.isEmpty()) context.write(new Text("product"), new LongWritable(Long.parseLong(s)));
    }
  }
  public static class ProductReducer extends Reducer<Text, LongWritable, Text, LongWritable> {
    public void reduce(Text key, Iterable<LongWritable> values, Context context)
        throws IOException, InterruptedException {
      long product = 1;
      for (LongWritable v : values) product *= v.get();
```

```
      context.write(new Text("Product"), new LongWritable(product));
    }
  }
  public static void main(String[] args) throws Exception {
    if (args.length != 2) {
      System.err.println("Usage: NumberProduct <input> <output>"); System.exit(2);
    }
    Job job = Job.getInstance(new Configuration(), "Multiply Two Numbers");
    job.setJarByClass(NumberProduct.class);
    job.setMapperClass(NumberMapper.class); job.setReducerClass(ProductReducer.class);
    job.setMapOutputKeyClass(Text.class); job.setMapOutputValueClass(LongWritable.class);
    job.setOutputKeyClass(Text.class); job.setOutputValueClass(LongWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
# numbers.txt contains:
6
7
hadoop com.sun.tools.javac.Main NumberProduct.java
jar cf product.jar NumberProduct*.class
hadoop jar product.jar NumberProduct /lab/numbers-in /lab/product-out
hdfs dfs -cat /lab/product-out/part-r-00000
```

Output

For input numbers 6 and 7, output is: Product 42.

Result

Thus, the two input numbers were multiplied using MapReduce.

Experiment 17: Add Two Numbers Using Hadoop MapReduce

Aim

To add two numbers using a MapReduce job.

Procedure

71. Save as NumberSum.java.
72. Prepare one number per input line.
73. Upload the file to HDFS.
74. Compile and run the job.
75. Read the output.

Program / Commands

```
// NumberSum.java
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class NumberSum {
  public static class NumberMapper extends Mapper<LongWritable, Text, Text, LongWritable> {
    public void map(LongWritable key, Text value, Context context)
```

```
        throws IOException, InterruptedException {
      String s = value.toString().trim();
      if (!s.isEmpty()) context.write(new Text("sum"), new LongWritable(Long.parseLong(s)));
    }
  }
  public static class SumReducer extends Reducer<Text, LongWritable, Text, LongWritable> {
    public void reduce(Text key, Iterable<LongWritable> values, Context context)
        throws IOException, InterruptedException {
      long sum = 0;
      for (LongWritable v : values) sum += v.get();
      context.write(new Text("Sum"), new LongWritable(sum));
    }
  }
  public static void main(String[] args) throws Exception {
    if (args.length != 2) {
      System.err.println("Usage: NumberSum <input> <output>"); System.exit(2);
    }
    Job job = Job.getInstance(new Configuration(), "Add Two Numbers");
    job.setJarByClass(NumberSum.class);
    job.setMapperClass(NumberMapper.class); job.setCombinerClass(SumReducer.class);
    job.setReducerClass(SumReducer.class);
    job.setMapOutputKeyClass(Text.class); job.setMapOutputValueClass(LongWritable.class);
    job.setOutputKeyClass(Text.class); job.setOutputValueClass(LongWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
# numbers.txt:
10
20
hadoop com.sun.tools.javac.Main NumberSum.java
jar cf sum.jar NumberSum*.class
hadoop jar sum.jar NumberSum /lab/numbers-in /lab/sum-out
hdfs dfs -cat /lab/sum-out/part-r-00000
```

Output

For input numbers 10 and 20, output is: Sum 30.

Result

Thus, the two numbers were added using MapReduce.

Experiment 18: Hive SELECT, GROUP BY, WHERE, HAVING, ORDER BY

Aim

To create an employee table and run the required Hive queries.

Procedure

76. Create the table and insert sample records.
77. Run SELECT for all fields.
78. Group by department.
79. Filter rows with WHERE and groups with HAVING.
80. Sort by employee number.

Program / Commands

```
CREATE TABLE employee (
```

```
  empno INT, ename STRING, department STRING, salary INT
);
INSERT INTO employee VALUES
(101, 'Ravi', 'CSE', 30000),
(102, 'Priya', 'AIML', 40000),
(103, 'Kumar', 'CSE', 35000);

SELECT * FROM employee;
SELECT department, COUNT(*) FROM employee GROUP BY department;
SELECT * FROM employee WHERE salary > 30000;
SELECT department, AVG(salary) AS avg_salary
FROM employee GROUP BY department
HAVING AVG(salary) > 32000;
SELECT * FROM employee ORDER BY empno;
```

Output

SELECT displays all records; GROUP BY returns counts by department; WHERE filters rows; HAVING filters groups; ORDER BY sorts employee numbers ascending.

Result

Thus, the required Hive queries were executed.

Experiment 19: HDFS File Management

Aim

To create, upload, display, download, and remove files in HDFS.

Procedure

81. Ensure HDFS is running.
82. Create a directory and upload a file.
83. List and display the file.
84. Download it to the local directory.
85. Remove the HDFS file and empty directory.

Program / Commands

```
hdfs dfs -mkdir -p /lab/input
hdfs dfs -put sample.txt /lab/input/
hdfs dfs -ls /lab/input
hdfs dfs -cat /lab/input/sample.txt
hdfs dfs -get /lab/input/sample.txt .
hdfs dfs -rm /lab/input/sample.txt
hdfs dfs -rmdir /lab/input
```

Output

Commands list, display, download, and remove the file and directory.

Result

Thus, HDFS file-management operations were performed.

Experiment 20: Hive UPDATE, EXPORT, IMPORT, SORT BY, DISTRIBUTE BY, CLUSTER BY

Aim

To demonstrate additional Hive commands and sorting/distribution operations.

Procedure

86. Create a transactional table if UPDATE is required; transaction support depends on Hive configuration.
87. Insert sample data.
88. Run UPDATE on the transactional table.
89. Export the table and import it to a new table/location using a supported Hive version.
90. Execute SORT BY, DISTRIBUTE BY, and CLUSTER BY queries.

Program / Commands

```
-- Example transactional table; exact ACID settings depend on Hive version
CREATE TABLE staff (
  eid INT, ename STRING, department STRING
)
STORED AS ORC TBLPROPERTIES ('transactional'='true');

INSERT INTO staff VALUES
(1, 'Ravi', 'CSE'), (2, 'Priya', 'AIML');
UPDATE staff SET department = 'AIML' WHERE eid = 1;

EXPORT TABLE staff TO '/user/hive/exports/staff_export';
IMPORT TABLE staff_copy FROM '/user/hive/exports/staff_export';

SELECT * FROM staff SORT BY department;
SELECT * FROM staff DISTRIBUTE BY department;
SELECT * FROM staff CLUSTER BY department;
```

Output

UPDATE modifies the selected row when ACID support is configured. EXPORT/IMPORT transfers table data and metadata. SORT BY sorts within reducers; DISTRIBUTE BY controls reducer distribution; CLUSTER BY combines distribution and sorting by the same expression.

Result

Thus, the specified Hive commands were demonstrated, subject to Hive version and configuration.

Quick Lab Exam Notes

- For Hadoop jobs, the HDFS output directory must not already exist; delete the old output directory before rerunning a job.
- The Java examples use Hadoop MapReduce APIs and should be compiled against the Hadoop libraries installed in your lab. A common alternative is to build them with Maven using the Hadoop version provided by the lab.
- MatrixMultiply assumes valid matrix dimensions and well-formed input records. The input records must use zero-based row and column indexes.
- WordCharacterCount emits each word occurrence and its character length; it does not aggregate repeated words into one unique word-length record.
- The NumberProduct and NumberSum examples expect numeric input, one number per line. NumberProduct multiplies every number supplied; for the lab example, supply exactly two numbers.
- Hive UPDATE requires a transactional table and compatible configuration. Thrift and HBase startup commands vary significantly by software version.
- Use the sample output as a reference. Actual timestamps, file sizes, service names, and formatting can differ on the lab machine.